

## PHP | Looping Statement

- PHP uses different types of loops to execute the same code continuously.
  - Loops help to decrease code repetition and optimize programs. Loops allow programmers to save time and effort by writing less code.
  - We use loops to avoid writing the same code repeatedly.
  - Loops are important in web applications for showing numbers, processing data and carrying out repetitive tasks.
  - The proper execution of loops allows programs to run faster.
- 

### ➤ Types of Loop

PHP supports following four loop types.

- **For Loop**: Loops through a block of code a specified number of times.
- **While Loop**: Loops through a block of code if and as long as a specified condition is true.
- **Do-while Loop**: Loops through a block of code once, and then repeats the loop as long as a special condition is true.

### 1. PHP for Loop

PHP for loop is used when you know exactly how many times you want to iterate through a block of code. It consists of three expressions:

**Initialization**: Sets the initial value of the loop variable.

**Condition**: Checks if the loop should continue.

**Increment/Decrement**: Changes the loop variable after each iteration.

**Syntax:**

```
for ( Initialization; Condition; Increment/Decrement ) {  
    // Code to be executed  
}
```

**Example 1:** Printing numbers from 1 to 5 using a for loop.

```
<?php
// Code to illustrate for loop
for ($num = 1; $num <= 5; $num += 1) {
    echo $num . "\n";
}
?>
```

**Output:**

```
1
2
3
4
5
```

**Example 2:**

```
<?php
$a = 0;
$b = 0;
for( $i = 0; $i<5; $i++ ) {
    $a += 10;
    $b += 5;
}
echo ("At the end of the loop a = $a and b = $b" );
?>
```

**Output:**

At the end of the loop a = 50 and b = 25

---

## 2. PHP while Loop

The while loop is also an entry control loop like for loops. It first checks the condition at the start of the loop, and if it's true then it enters into the loop and executes the block of statements and goes on executing it as long as the condition holds true.

### Syntax:

```
while ( condition ) {  
    // Code is executed  
}
```

**Example 1:** Printing numbers from 1 to 5.

```
<?php  
$num = 1;  
while ($num <= 5) {  
    echo $num . "\n";  
    $num++;  
}  
?>
```

### Output:

```
1  
2  
3  
4  
5
```

**Example 2:**

```
<?php  
$i = 0;
```

```
$num = 50;
while($i < 10) {
    $num--;
    $i++;
}
echo ("Loop stopped at i = $i and num = $num" );
?>
```

**Output:**

Loop stopped at i = 10 and num = 40

---

### 3. PHP do-while Loop

The do-while loop is an exit control loop, which means, it first enters the loop, executes the statements, and then checks the condition. Therefore, a statement is executed at least once using the do...while loop. After executing once, the program is executed as long as the condition holds true.

**Syntax:**

```
do {
    // Code is executed
} while ( condition );
```

**Example 1:** Printing numbers from 1 to 5.

```
<?php
$num = 1;
do {
    echo $num . "\n";
    $num++;
}
```

```
} while ($num <= 5);
```

```
?>
```

**Output:**

1

2

3

4

5

**Example 2:**

```
<?php
```

```
    $i = 0;
```

```
    $num = 0;
```

```
    do {
```

```
        $i++;
```

```
    }
```

```
    while( $i < 10 );
```

```
    echo ("Loop stopped at i = $i" );
```

```
?>
```

**Output:**

Loop stopped at i = 10